Surname (Cognome)	
Name (Nome)	
POLIMI ID Number	
Signature (Firma)	

Politecnico di Milano, July 9, 2018

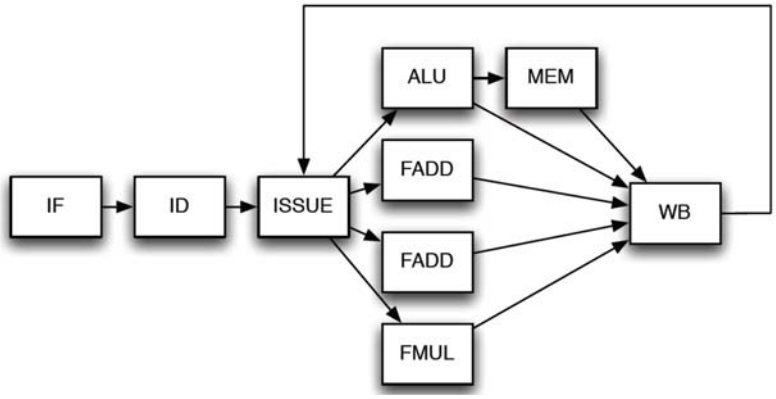
Course on Advanced Computer Architectures

Prof. D. Sciuto, Prof. C. Silvano

EX1	**(5 points)**	
EX2	**(5 points)**	
EX3	**(6 points)**	
Q1	**(5 points)**	
Q2	**(5 points)**	
SIX QUESTIONS	**(6 points)**	
TOTAL	**(32 points)**	

EXERCISE 1 – MULTI-CYCLE PIPELINE (5 points)

In this problem we will examine the execution of the following code segment on the following single-issue out-of-order processor:



- All functional units are pipelined
- ALU operations take 1 clock cycle
- Memory operations take 2 clock cycles (includes time in ALU)
- FP ALU operations take 2 clock cycles
- The Write Back unit has a single write port
- There is no register renaming
- Instructions are fetched, decoded and issued in order
- An instruction will only enter the ISSUE stage if it does not cause a WAR or WAW hazard
- Only one instruction can be issued at a time, and in the case multiple instructions are ready, the oldest one will go first

Fill in the following table pointing out, for each instruction, the pipeline stages activated in each clock cycle and **type of hazards:**

Instruction	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	Type of Hazards
`lw.d $f1,BASEA($r1)`													
`addi.d $f1,$f1,k1`													
`add.d $f3,$f2,$f1`													
`add.d $f1,$f2,$f3`													
`sw.d $f1,BASEA($r1)`													

Insert in the empty pipeline scheme the **pipeline stages** and **stalls** needed to solve the previous data, control and structural hazards.

INS	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21
I1																					
I2																					
I3																					
I4																					
I5																					

Fill in the following table pointing out the **NUMBER OF STALLS** to be inserted **within** each instruction to solve the hazards:

Num. of Stalls	INSTRUCTION	Type of Hazards
	`lw.d $f1,BASEA($r1)`	
	`addi.d $f1,$f1,k1`	
	`add.d $f3,$f2,$f1`	
	`add.d $f1,$f2,$f3`	
	`sw.d $f1,BASEA($r1)`	

EXERCISE 2 – TOMASULO (5 points)

Assume that the following code has been executed on a CPU with dynamic scheduling based on TOMASULO algorithm.

1. List all the possible conflicts in the code (use the column **HAZARDS TYPE**):

Instruction	Issue	Execution Start	Write Result	HAZARDs TYPE
I1: LW.D $F6, 32(R2)	1	2	8	
I2: ADDD $F2, $F6, $F4	2	9	11	
I3: MULTD $F0, $F4, $F2	3	12	32	
I4: SUBD $F12, $F2, $F6	12	13	15	
I5: ADDD $F0, $F12, $F2	16	17	19	

2. Is there a “hardware configuration” that can respect the shown execution timing?

3. If it is possible, describe the architecture in terms of Functional Units and Reservation Stations: List the number, type and latency of Functional Units and the number of Reservation Stations for each FU

EXERCISE 3: VLIW (6 points)

In this problem, you will port code to a simple **3-issue VLIW machine,** and schedule it to improve performance. Details about the 3-issue VLIW machine with 3 fully pipelined functional units:

- Integer ALU with **1** cycle latency to next Integer/FP and **2** cycle latency to next Branch
- Memory Unit with **3** cycle latency
- Floating Point Unit with **3** cycle latency (each FPU can complete one add or one multiply per clock cycle)
- Branch completed with **1 cycle delay slot** (branch solved in ID stage)
- No interlocks

C Code:

```
for(int i=0; i<N; i++)
     A[i] = A[i] + B[i];
```

Assembly Code:

```
loop:ld   f1, 0(r6)
     ld   f2, 0(r6)
     fadd f2, f2, f1
     st   f2, 0(r6)
     addi r6, r6, 4
     bne  r6, r7, loop
```

ESERCISE 3.A

Considering **one iteration** of the loop, **schedule** the assembly code for the 3-issue VLIW machine in the following table by using the **list-based scheduling,** but neither using software pipelining nor loop unrolling **nor modifying loop indexes.** Please do not need to write in NOPs (can leave blank).

	Integer ALU	Memory Unit	FPU
C0			
C1			
C2			
C3			
C4			
C5			
C6			
C7			
C8			
C9			
C10			
C11			
C12			
C13			

How long is the Critical Path?_____

What performance did you achieve in FP ops per cycle?_____

What performance did you achieve in cycles per loop iteration?_____

What code efficiency did you achieve?_____

What CPI did you achieve? _____

EXERCISE 3.B

Considering the unrolling of two iterations of the loop as follows:

Assembly Code:

```
loop:ld   f1, 0(r6)
     ld   f2, 0(r6)
     fadd f2, f2, f1
     st   f2, 0(r6)
     ld   f3, 4(r6)
     ld   f4, 4(r6)
     fadd f4, f4, f3
     st   f4, 4(r6)
     addi r6, r6, 8
     bne  r6, r7, loop
```

Considering **one iteration of the unrolled loop,** schedule the assembly code by using the **list-based scheduling,** but neither using software pipelining nor loop unrolling **nor modifying loop indexes.** Please do not need to write in NOPs (can leave blank).

	Integer ALU	Memory Unit	FPU
C0			
C1			
C2			
C3			
C4			
C5			
C6			
C7			
C8			
C9			
C10			
C11			
C12			
C13			
C14			
C15			

How long is the Critical Path?_____

What performance did you achieve in FP ops per cycle? _____

What performance did you achieve in cycles per loop iteration?_____

What code efficiency did you achieve? _____

What CPI did you achieve? _____

How much is the Speedup with respect to the previous EXE 3.A?

_____

QUESTION 1: (5 points)

1. Superscalar processors today use an approach that exploits both **Instruction Level Parallelism** and **Thread Level Parallelism.** Briefly explain the approach

2. Which **HW mechanisms** of dynamic scheduled processors intrinsically support this approach?

3. Which **modifications** to a generic architecture of dynamically scheduled superscalar processors are required?

QUESTION 2: (5 points)

1. Explain the concepts of the **Reorder Buffer** and the motivations to introduce it in dynamically superscalar architectures.

2. In **Speculative Tomasulo** how is the **Reorder Buffer** used and during which stage the entry for the ROB is assigned?

SIX QUESTIONS: (6 points)

Answer with **True** or **False** to the following questions:

Q1) *In MIMD architectures with a physically centralized memory organization, the memory address space is shared*

Answer: **T** **F**

Q2) *Increasing the issue rate of modern processors (for instance up to 12 instructions per clock cycle) provides a huge improvement in the processor clock frequency*

Answer: **T** **F**

Mark the **True** answers to the following questions (some questions might have **multiple True answers):**

Q3) *Considering the MESI – write-invalidate write-back protocol, a cached block can be in one of four states: Modified, Exclusive, Shared, Invalid. For which of these states is the memory up to date?*

Answer 1: Only Exclusive. **T**
Answer 2: Only Shared. **T**
Answer 3: Shared and Exclusive. **T**
Answer 4: Shared and Modified. **T**

Q4) *In case of delayed branch technique, which of the following assertions are true?*

Answer 1: Instruction in the branch delay slot is always executed **T**
Answer 2: Instruction in the branch delay slot is always a NOP **T**
Answer 3: Instruction in the branch delay slot is always an independent instruction from before the branch **T**
Answer 4: Helps the compiler to statically predict the outcome of the branch **T**

Q5) Which complications are introduced by out-of-order execution and out-of-order completion?

Answer 1: WAR and WAW hazards **T**
Answer 2: RAW hazards **T**
Answer 3: Exception handling **T**
Answer 4: Control hazards **T**
Answer 5: Instruction reordering **T**

Q6) Which of these modifications would you introduce on a cache to decrease the **conflict misses**?

Answer 1: Increase the cache size **T**
Answer 2: Increase the cache associativity **T**
Answer 3: Increase the size of the blocks **T**
Answer 4: Add a second level of cache **T**